

Predicción de resultados del fútbol uruguayo mediante aprendizaje supervisado

Cecilia Waksman, Lucia Coudet

Aprendizaje Automático 2026

Facultad de Ingeniería, Udelar

Montevideo, Uruguay

Resumen—Se aborda la predicción del resultado de partidos del fútbol uruguayo según gane el cuadro local, el cuadro visitante, o haya empate, a partir del conjunto de datos de David Schoch, restringido a los partidos del fútbol uruguayo entre los años 1932–2025. Se construye un pipeline reproducible de preprocesamiento y *feature engineering* temporal y causal. Se implementa de manera manual un Árbol de decisión y un clasificador Naive Bayes. Asimismo, se implementa un Clasificador Base que prediga que gana el equipo que tiene mayor proporción de partidos ganados en los últimos 10 años. Se comparan los resultados obtenidos con los de los clasificadores de la librería *scikit-learn* de python, incluyendo también el método de Random Forest. Se obtiene que en términos de *accuracy* el Clasificador Base tiene el mejor desempeño. Luego se realiza un ajuste con técnicas de validación cruzada para el tuneo de hiperparámetros y se compara el desempeño de todos los clasificadores implementados. Se computan las medidas de *accuracy*, *recall*, *F1* por clase, y macro *F1*. En términos de *accuracy* se concluye que Random Forest tiene el mejor desempeño predictivo, mientras que según el indicador macro *F1* la implementación Naive Bayes manual lo tiene. El modelo con menor desempeño en ambas medidas es la implementación manual del Árbol de decisión seguido por la implementación de *scikit learn* del mismo.

Index Terms—aprendizaje supervisado, fútbol, árbol de decisión, Naive Bayes, series temporales

I. INTRODUCCIÓN

Predecir el resultado de un partido de fútbol es un problema clásico de clasificación multiclase con tres salidas: gana el local (L), hay empate (E) o gana el visitante (V). El fútbol posee un fuerte componente aleatorio, por lo que incluso modelos complejos muchas veces no logran superar las reglas más sencillas o intuitivas. En este trabajo se utiliza el conjunto de datos público de resultados de ligas de David Schoch [1], restringido a los partidos del fútbol uruguayo entre 1932 y 2025 (15.207 registros).

Dicho conjunto contiene información sobre el nombre del cuadro local, el nombre del cuadro visitante, la fecha en que se jugó el partido, el número de goles del cuadro local, el número de goles del cuadro visitante, y un atributo que indica si el partido terminó a los 90 minutos, si hubo tiempo extra, o si hubo penales (*full_time*).

También contiene información como ser el identificador, país, código, y continente del cuadro local y del cuadro visitante, y el continente donde se realizó la competición.

Los objetivos son: (i) construir un pipeline reproducible de preprocesamiento; (ii) implementar clasificadores super-

visados, incluyendo dos algoritmos propios; y (iii) evaluar el desempeño con un protocolo temporal riguroso que evite fugas de información, implementando técnicas de validación cruzada.

II. PREPROCESAMIENTO

El preprocesamiento implementado consiste en la realización de la limpieza de los datos y de la transformación de los mismos a un formato aceptable para los algoritmos a utilizar.

En lo que respecta a la limpieza, se verifica que las columnas no tienen errores y que no hay valores faltantes en ningún atributo. Asimismo, se eliminan filas duplicadas. También se verifica que los años de las fechas de los partidos se encuentran dentro del rango esperado: 1932–2025.

En lo que respecta a la transformación de los datos, se definieron los tipos de atributos (numéricos, categóricos, fechas) y se eliminaron los atributos constantes que no aportan información a los algoritmos: país, código, y continente del cuadro local y del cuadro visitante, y el continente donde se realizó la competición, así como también aquellos atributos que aportan exactamente la misma información que algún otro: identificadores del cuadro local y del cuadro visitante.

Para evitar la fuga de información (*data leakage*), se elimina el atributo *full_time*, debido a que contiene información que no es posible conocer antes de que el partido termine. Introduce una correlación con el resultado del tiempo regular, ya que para que exista tiempo extra o penales es necesario que el resultado en tiempo regular haya sido empate. A su vez, conocer que un partido se define por penales permite descartar que el resultado sea empate. Incluir este atributo le otorga al algoritmo una pista directa del desarrollo del encuentro que invalida su capacidad predictiva real para futuros partidos, cuando el objetivo es obtener la predicción antes de que comiencen los mismos. En la medida que es información que no se tiene a priori, no sería posible utilizar el modelo para predecir nuevos resultados si se mantuviera este atributo en el conjunto de entrenamiento.

En cuanto a la creación de nuevos atributos, se generaron a partir del atributo *fecha*: el año, el mes, y el día de la semana en el que se llevó a cabo el partido, y un atributo indicatriz que señala si fue realizado un fin de semana.

También se crearon los atributos *tasa de victorias local* y *tasa de victorias visitante*, que indican la tasa de victorias del cuadro local y visitante respectivamente, considerando una

ventana móvil de los últimos 5 partidos, y la diferencia entre ellas, *diferencia de tasas de victoria*.

Finalmente se crea el atributo objetivo *resultado* comparando los goles del cuadro local y los goles del cuadro visitante, siendo los posibles resultados: gana el cuadro local (L), gana el cuadro visitante (V) o hay empate (E).

Una vez creados los nuevos atributos definidos anteriormente y realizada la limpieza mencionada, se realiza una serie de transformaciones a los datos según el tipo de columna y el algoritmo en que será utilizado:

- Para la implementación del Árbol de Decisión (tanto manual como de scikit learn) y del Random Forest, a las columnas categóricas se le aplica one-hot encoding utilizando la función *OneHotEncoder* de la librería *scikit learn*. A las columnas numéricas se las mantiene tal como están, ya que se verifica que todas pertenecen al intervalo 0-1 (no es necesario escalar).
- Para la implementación del método de Naive Bayes de *scikit learn*, a cada valor de los atributos, tanto numéricos como categóricos, se le asigna un valor entero utilizando la función *OrdinalEncoder* de *scikit learn*. En este paso se incluyó también los atributos numéricos *tasa de victorias local* y *tasa de victorias visitante*, porque estos toman relativamente pocos valores distintos, pero no así *diferencia de tasas de victoria*, porque toma muchos valores distintos que se repiten pocas veces. Esta transformación es necesaria, ya que el método Naive Bayes Categórico asume que los datos provienen de atributos discretos (categorías fijas) para contar frecuencias absolutas y calcular las probabilidades condicionales.
- Para la implementación del método de Naive Bayes manual las atributos numéricos *tasa de victorias local* y *tasa de victorias visitante* se transforman en categóricos y se utilizan estas dos sumado al resto de atributos categóricos con las que ya contabamos.

III. MODELOS SUPERVISADOS

III-A. Partición de la muestra

Para la implementación de los modelos de aprendizaje supervisado, se realizó una partición de la muestra en conjunto de entrenamiento y conjunto de evaluación que considera la temporalidad de los datos, en particular en lo que respecta al orden cronológico de los mismos, de manera de asegurar que para realizar predicción en un determinado momento del tiempo se utilizó información disponible solamente hasta ese momento del tiempo, evitando así la fuga de información (*data leakage*). De esta manera, se consideraron los partidos jugados hasta el año 2023 inclusive como conjunto de entrenamiento y los jugados en 2024 y 2025 como conjunto de evaluación.

III-B. Feature selection

Con el fin de seleccionar los atributos con mayor capacidad predictiva, se realiza la selección de los atributos más importantes calculando la información mutua (*mutual information*) entre cada atributo del conjunto de datos y el atributo objetivo.

La información mutua intenta medir cuánta incertidumbre sobre la clase objetivo se reduce al conocer el valor de cada atributo. Un valor de 0 indica independencia total mientras que valores más altos reflejan una relación de dependencia más fuerte. Su cálculo es similar al de la Ganancia definida mediante Entropía (vista en clase).

Para la codificación de los atributos categóricos se utiliza la función *OrdinalEncoder* que transforma las categorías en números enteros.

Se decide eliminar los atributos que aportan una ganancia de información inferior a 0.01. Estos fueron el atributo que indica el día de la semana en que se jugó el partido y el atributo indicatriz que señala si fue fin de semana o no.

III-C. Árbol de decisión manual

Para la implementación propia del árbol de decisión con ganancia mínima de información se utiliza la técnica de one-hot encoding en el preprocesamiento, de forma de convertir los atributos categóricos en nuevas columnas binarias.

Durante el entrenamiento, el algoritmo construye el árbol de forma recursiva dividiendo los datos en cada nodo según el atributo y umbral que maximizan la ganancia de información (basada en la reducción de entropía). Maneja tanto atributos numéricos como categóricos. El crecimiento se detiene al alcanzar una profundidad máxima o cuando la ganancia cae por debajo de la ganancia mínima de información especificada, asignando en las hojas la clase mayoritaria. En este caso, se le especificó una ganancia mínima de información de 0.01 y no se limitó a una profundidad máxima.

En la parte de predicción, se evalúan nuevas observaciones recorriendo la estructura desde la raíz hasta la hoja correspondiente para retornar la clase predicha.

A continuación se presenta la matriz de confusión obtenida:

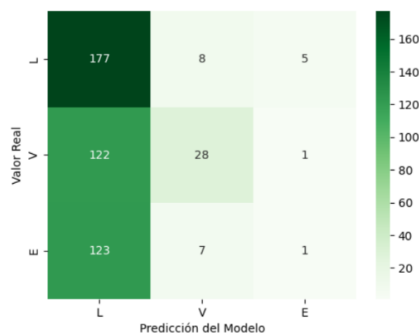


Figura 1. Matriz de confusión del Árbol de decisión manual

En la amplia mayoría de casos este modelo predice que gana el equipo local, por eso el recall para esta categoría es bastante alto y para el resto de ellas es muy bajo. En general, el modelo predijo correctamente el 43 % de las instancias de testeo.

III-D. Árbol de decisión de scikit learn

Por otro lado, se realizó la implementación del árbol de decisión mediante el clasificador de Scikit Learn

DecisionTreeClassifier, con los siguientes parámetros: *min_impurity_decrease* = 0.01, *max_depth* = None, *criterion* = Entropy, *min_samples_leaf* = 100.

Observando la matriz de confusión se verificó que la clase "Hay empate" tiene precisión y recall 0. Esto puede deberse a:

- Desbalance severo de clases: La clase con 0 accuracy representa un porcentaje muy pequeño del conjunto de datos y el árbol la ignora para maximizar el accuracy global.
- El árbol está sobreajustado o simplificado en exceso
- Métrica de optimización incorrecta por utilizar el accuracy como criterio de optimización, el cual prioriza a las clases con más muestras.

Se propone balancear el peso de las clases utilizando el argumento *class_weight='balanced'*, esto le da más importancia (penalización) a los errores cometidos en las clases con menos muestras. A continuación se presenta la matriz de confusión obtenida en este ajuste:

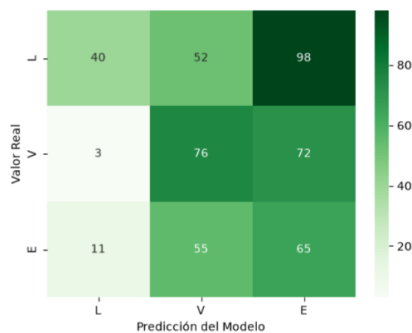


Figura 2. Matriz de confusión del Árbol de Decisión scikit learn balanceado

Se observa que el modelo balanceado predice proporciones parecidas de las tres categorías posibles de respuesta. Según el recall, este modelo predice la mitad de las instancias que efectivamente cuentan con las categorías "Gana visitante" y "Empate" como tales y solo el 20 % de las que efectivamente son "Gana local" como tales. El Árbol de scikit learn sin balanceo tiene un Accuracy mayor que con balanceo, 0.45 contra 0.38 respectivamente.

III-E. Naive Bayes manual

Se implementa de manera manual un clasificador Naive Bayes Categórico compatible con el de scikit-learn. En el entrenamiento, calcula las probabilidades a priori de las clases y las condicionales utilizando el m-estimador para suavizar probabilidades de valores ausentes o sin frecuencia. En el predict, suma los logaritmos de estas probabilidades por fila para seleccionar la clase con mayor valor acumulado. Por último, entrena el modelo con parámetro *m* = 1 y se evalúan sus predicciones imprimiendo la métrica de Accuracy y el Classification Report. A continuación se presenta la matriz de confusión obtenida:

Este modelo predice correctamente el casi 46 % de las instancias de testeo y tiende a predecir mayormente que gana el equipo local, por lo que el recall de esta categoría es

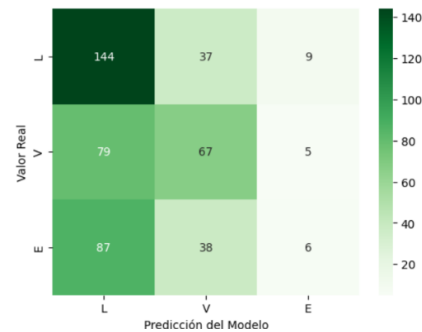


Figura 3. Matriz de confusión Naive Bayes manual

relativamente alto. Aún así, su precisión, similar a la de las demás categorías, está por debajo del 0.50, es decir que las instancias que se predicen dentro de esta categoría en mayor proporción no pertenecen a la misma. Las métricas de la categoría "Empate" son en general bastante bajas en este modelo.

III-F. Naive Bayes scikit learn

También se implementó el ajuste utilizando el clasificador de la librería Scikit Learn *CategoricalNB* con parámetros *alpha=1* (Suavizado de Laplace. Como el m-estimador, es un parámetro de suavizado para cuando existen categorías con una proporción de datos cercana a 0) y *fit_prior = True*. La matriz de confusión obtenida fue la siguiente:

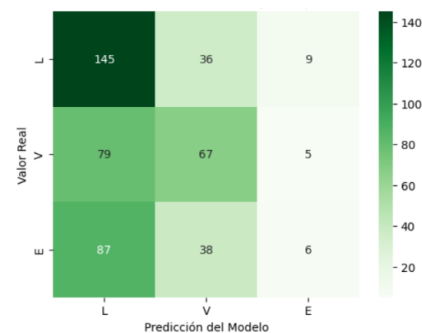


Figura 4. Matriz de confusión Naive Bayes scikit learn

La implementación del Naive Bayes con scikit learn predice de forma muy similar a la implementación manual las categorías de respuesta y también cuentan con métricas muy parecidas, prediciendo la implementación con scikit learn apenas un 1 % mejor que la manual.

III-G. Clasificador base

Se implementó un clasificador base que en el entrenamiento identifica la clase mayoritaria global y calcula la tasa histórica de victorias de cada equipo considerando únicamente los últimos 10 años. En el predict, compara la tasa de victorias del equipo local contra el visitante: predice local (L) o visitante (V) según cuál tenga mayor tasa, y asigna la clase mayoritaria en caso de empate o falta de datos. Finalmente, ajusta el modelo con los datos de entrenamiento y evalúa sus

predicciones en el conjunto de prueba, imprimiendo la medida de Accuracy y Classification Report. En este caso también se observa que la clase "Hay empate" tiene precision y recall 0.

El modelo balancea mejor sus predicciones entre "Gana local" y "Gana visitante" y su capacidad de predicción entre los modelos implementados (previo al tuneo de hiperparámetros) es la mejor según el Accuracy.

III-H. Random Forest scikit learn

Por último, se realizó un ajuste mediante el método de Random Forest utilizando el clasificador de Scikit Learn *RandomForestClassifier* con parámetros $n_estimators=300$, $max_depth=None$, $min_samples_leaf=10$, $max_features="sqrt"$. e obtuvo la siguiente matriz de confusión:

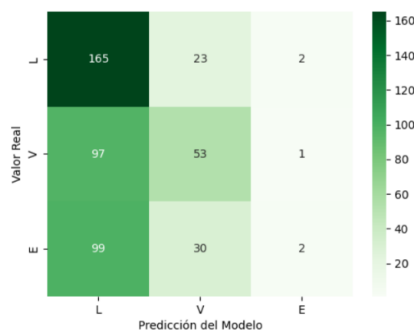


Figura 5. Matriz de confusión Random Forest scikit learn

De forma similar a alguno de los modelos ya implementados, el Random Forest tiende a predecir mayormente que "Gana local" y cuenta con un Accuracy de casi 0.47.

III-I. Comparación de modelos

Si bien es necesario tener en cuenta todas las medidas de bondad de ajuste para realizar la comparativa de modelos, a modo de resumen se presenta el gráfico con el Accuracy obtenido en cada uno de los métodos comentados en las subsecciones anteriores.

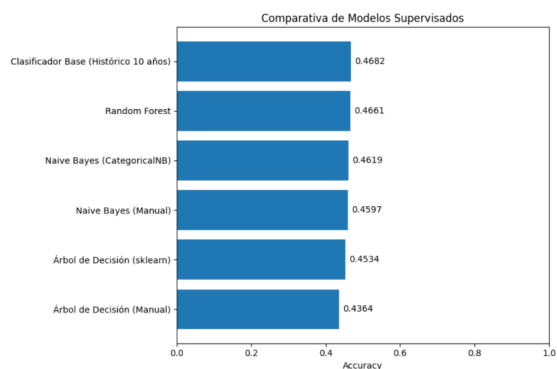


Figura 6. Accuracy de los modelos ajustados

Como puede observarse, todos los modelos cuentan con valores de Accuracy relativamente cercanos. El Clasificador

Base es el que tiene el mejor desempeño en términos de esta medida, mientras que el Árbol de Decisión manual el peor. No obstante si comparamos con otras medidas los resultados pueden diferir.

IV. EXPERIMENTACIÓN

En la etapa de experimentación, se definió un pipeline de preprocesamiento para cada modelo según los atributos a utilizar en cada algoritmo y las transformaciones específicas necesarias en cada uno de ellos.

IV-A. Tratamiento de la temporalidad

Sobre el conjunto de entrenamiento definido anteriormente, que contiene información de los partidos jugados hasta el año 2023 inclusive, se definió función de validación cruzada con ventanas móviles de 1 año, las cuales definen los grupos utilizados para validar. Se definieron 5 grupos de 1 año cada uno usando los datos desde 2019 a 2023 inclusive, y para cada uno de ellos se entrenan los modelos con los datos de los partidos jugados en fechas anteriores.

Asegurarse de no utilizar datos futuros para predecir datos pasados es un aspecto importante en el tratamiento de la temporalidad en los modelos de aprendizaje supervisado, ya que evita el sesgo prospectivo. La utilización del método de validación cruzada tradicional (k-folds) que mezcla observaciones al azar no es adecuado para datos de este tipo.

IV-B. Ajuste de hiperparámetros

Se definió una grilla de valores para cada hiperparámetro de cada algoritmo y se observó la evolución del error promedio para cada valor posible de cada hiperparámetro en el conjunto de validación. Se seleccionaron como mejores hiperparámetros los que en conjunto minimizan el error ($1 - Accuracy$).

Una vez obtenidos los mejores hiperparámetros, dentro de los especificados en cada grilla propuesta, se reentrenó cada modelo sobre todo el conjunto de entrenamiento.

IV-C. Resultados en evaluación

Finalmente, se realizó la evaluación de cada modelo sobre el conjunto de testeo y se compararon las matrices de confusión y las métricas accuracy, precision, recall, F1 por clase, y macro-F1.

A modo de resumen, a continuación se presenta la tabla de resultados con las valores del Accuracy obtenido y macro F1:

modelo	accuracy	macro_f1
Random Forest	0.48093	0.35854
Clasificador base - histórico 10 años	0.46822	0.36128
CategoricalNB	0.46186	0.36922
Naive Bayes manual	0.45975	0.37094
Árbol sklearn	0.45763	0.34763
Árbol manual	0.45551	0.34613

Figura 7. Tabla de resultados en evaluación

La tabla presenta una comparativa de desempeño entre los seis modelos de clasificación evaluados. En términos de

acierto global, Random Forest encabeza los resultados con un Accuracy de 0.48093, superando al resto de los enfoques. Sin embargo, al observar la métrica Macro F1-Score, el modelo Naive Bayes manual obtiene el valor más alto con 0.37094, seguido muy de cerca por CategoricalNB con 0.36922, lo que sugiere un mejor equilibrio al predecir clases menos frecuentes como los empates. Por su parte, el Clasificador Base - histórico 10 años logra un rendimiento notablemente competitivo, situándose en el segundo puesto de Accuracy con 0.46822, superando tanto a los árboles individuales como a las variantes de Naive Bayes en tasa total de aciertos, y manteniendo un Macro F1 relativamente competitivo, de 0.36128. Finalmente, la comparativa evidencia una alta consistencia entre las implementaciones nativas y las desarrolladas manualmente: la versión propia de Naive Bayes registra métricas casi idénticas a las de CategoricalNB, mientras que el Árbol sklearn (0.45763 en Accuracy y 0.34763 en Macro F1) exhibe un comportamiento prácticamente equivalente al Árbol manual (0.45551 y 0.34613, respectivamente), quedando ambos modelos basados en un único árbol en los últimos lugares de desempeño.

V. CONCLUSIONES

En este trabajo se abordó la predicción de resultados del fútbol uruguayo bajo un esquema temporal riguroso que previno la fuga de información mediante el uso de validación cruzada con ventanas móviles. Los resultados confirman la elevada naturaleza estocástica del deporte, donde modelos más complejos rara vez superan de forma amplia a heurísticas simples.

A través de la experimentación se demostró que Random Forest obtuvo la mayor desempeño global (Accuracy de 0.4809), mientras que las variantes de Naive Bayes alcanzaron un mejor desempeño balanceado entre clases (Macro F1 superior a 0.3700) al capturar de forma más equitativa categorías complejas como los empates. Resulta destacable la competitividad del Clasificador Base basado en el histórico de 10 años, el cual superó en capacidad de pronóstico a los modelos de árboles individuales y a Naive Bayes, validando el peso de la historia reciente de los equipos como predictor directo.

Asimismo, se verificó la correcta construcción de los algoritmos propios: tanto las implementaciones manuales del Árbol de Decisión como de Naive Bayes exhibieron un comportamiento prácticamente idéntico a sus equivalentes de la librería scikit-learn.

Todo el desarrollo de este trabajo, incluyendo el código de las implementaciones manuales, el preprocesamiento y el análisis de resultados, se encuentra disponible de forma pública y reproducible en el siguiente [repositorio de GitHub](#)

Como líneas de trabajo futuro, se propone incorporar características dinámicas adicionales (como métricas avanzadas de xG o lesiones) y explorar técnicas de ensamblado o remuestreo para mitigar de forma más efectiva el desbalance intrínseco de la clase "Empate".

VI. USO DE INTELIGENCIA ARTIFICIAL

Para la realización de este trabajo utilizamos ChatGPT como herramienta de apoyo en la implementación y revisión de código, la resolución de errores y la comprensión de conceptos. Las soluciones propuestas por la herramienta fueron revisadas, entendidas y validadas antes de su incorporación al trabajo.

REFERENCIAS

- [1] D. Schoch, "football-data," repositorio de datos de resultados de fútbol. [En línea]. Disponible: <https://github.com/schochastics/football-data/tree/master>
- [2] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.